National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

CS-351 Compiler Construction
Project Report

Submitted by:

Qurratulain Zafar - 412655

Hamza Riaz - 414577

Muniba Noor - 407670

Project: Mini C Compiler

Implementation Language	C++17
Compiler Name	minic
Input	Mini C source file (.c)
Main Output	Tokens, Symbol Table, Three Address Code, Diagnostics
Compiler Type	Frontend compiler for a subset of C

Table of Contents:

1. Introduction and Objectives

This project is a Mini C Compiler Frontend implemented in C++17. It is designed to process a simplified subset of the C programming language and demonstrate the main frontend phases of compiler construction. The compiler reads a C-like source file, converts the source code into tokens, parses the token stream, performs semantic checks, maintains a symbol table, and produces Three Address Code (TAC) as an intermediate representation.

The project focuses on the analysis phase of compilation rather than final machine-code generation. Therefore, it does not produce assembly language or an executable version of the input program. Instead, it proves that the source program is lexically, syntactically, and semantically valid, and then displays the internal compiler outputs.

1.1 Objectives

- To implement lexical analysis for a defined Mini C language subset.
- To design a recursive-descent parser that recognizes declarations, statements, functions, and expressions.
- To perform semantic analysis using a symbol table and type system.
- To detect common programming errors such as undeclared identifiers, duplicate declarations, invalid assignments, and type mismatches.
- To generate Three Address Code as an intermediate representation of the program.
- To implement optional optimization features such as constant folding and TAC dead-code elimination.
- To validate the compiler using both accepted and rejected test programs.

2. Clearly Defined Subset of C

The compiler supports a compact and clearly limited subset of the C language. The subset is large enough to demonstrate major compiler-construction concepts, but intentionally avoids complex C features such as pointers, structures, unions, preprocessor expansion, and full standard-library support.

2.1 Supported Data Types

- int
- float
- char
- bool
- void
- string literals for lexical and semantic checking
- integer arrays such as int values[4]

2.2 Supported Language Features

2.3 Innovation Features

- Array support: The compiler supports single-dimensional integer arrays, including declarations, indexing, and assignment.

- Constant folding: Numeric literal expressions are evaluated during compilation to reduce unnecessary TAC operations.
- Dead code elimination: The TAC module removes unreachable instructions after unconditional return and goto statements until the next label or function boundary.

Feature	Description
Variables	Global and local variable declarations are supported.
Expressions	Arithmetic, relational, equality, logical, unary, assignment, postfix, and primary expressions are parsed.
Control Flow	if-else, while, and for statements are supported.
Functions	Functions may have parameters and return statements.
Scopes	Global, function, and block-level scopes are managed through the symbol table.
Arrays	Integer arrays can be declared, assigned, and accessed using indices.
Intermediate Code	The compiler emits Three Address Code using temporary variables and labels.
Constant Folding	Numeric literal expressions are evaluated during compilation where possible.

3. Grammar Specification

The parser follows a recursive-descent design, where major grammar rules are represented by separate parsing functions. The grammar below summarizes the Mini C subset accepted by the compiler. It is written in an EBNF-like format for readability.

```
program         -> external_declaration* EOF

external_decl    -> type_specifier identifier function_or_variable
function_or_var  -> function_definition | variable_declaration_rest

function_definition
                -> '(' parameter_list? ')' compound_statement

parameter_list   -> parameter (',' parameter)*
parameter        -> type_specifier identifier array_suffix?

variable_declaration
                -> type_specifier identifier declarator_rest (',' identifier declarator_rest)* ';'
declarator_rest  -> array_suffix? ('=' expression)?
array_suffix     -> '[' int_literal ']'

type_specifier   -> 'int' | 'float' | 'char' | 'bool' | 'void'

compound_statement
                -> '{' statement* '}'

statement        -> compound_statement
```

```
| if_statement
| while_statement
| for_statement
| return_statement
| local_declaration
| expression_statement
| ';'

if_statement     -> 'if' '(' expression ')' statement ('else' statement)?
while_statement  -> 'while' '(' expression ')' statement
for_statement    -> 'for' '(' for_init? ';' expression? ';' expression? ')' statement
return_statement -> 'return' expression? ';'
local_declaration-> variable_declaration
expression_stmt  -> expression ';'
```

3.1 Expression Grammar and Precedence

Expression parsing is divided into precedence levels. This ensures that expressions such as 2 + 3 * 4 are interpreted correctly as 2 + (3 * 4).

```
expression      -> assignment
assignment      -> logical_or ('=' assignment)?
logical_or      -> logical_and ('||' logical_and)*
logical_and     -> equality ('&&' equality)*
equality        -> relational (('==' | '!=') relational)*
relational      -> additive (('<' | '<=' | '>' | '>=') additive)*
additive        -> multiplicative (('+' | '-') multiplicative)*
multiplicative  -> unary (('*' | '/' | '%') unary)*
unary           -> ('!' | '-' | '++' | '--') unary | postfix
postfix         -> primary ('[' expression ']' | '++' | '--')*
primary         -> literal | identifier | function_call | '(' expression ')'
function_call   -> identifier '(' argument_list? ')'
argument_list   -> expression (',' expression)*
```

4. Design of Compiler Phases

The compiler is organized into standard frontend phases. Each phase has a clear responsibility and passes its output to the next phase.

Compiler Phase	Purpose
1. File Reading	The main driver reads the Mini C source file into a string.
2. Lexical Analysis	The lexer converts characters into tokens and reports invalid characters or malformed literals.
3. Syntax Analysis	The parser checks whether the token sequence follows the Mini C grammar.
4. Semantic Analysis	The parser and type system check declarations, scopes, assignments, conditions, returns, and function calls.
5. Symbol Table Management	Identifiers are stored with kind, type, return type, parameter list, and scope level.

6. TAC Generation	The compiler emits Three Address Code using temporary variables and labels.
7. Diagnostics	Errors are collected with line and column information and printed at the end.

4.1 Lexical Analysis

The lexer reads source code character by character and produces a stream of tokens. It recognizes keywords, identifiers, numeric literals, character literals, string literals, operators, brackets, braces, parentheses, commas, and semicolons.

- Skips whitespace and comments.
- Skips preprocessor-style lines beginning with #.
- Reports unexpected characters.
- Tracks line and column numbers for accurate error messages.

4.2 Syntax Analysis

The parser uses recursive-descent parsing. Each major grammatical category is handled by a separate parser function, such as parseStatement, parseIf, parseWhile, parseFor, parseReturn, and expression parsing functions.

- Checks top-level declarations.
- Parses functions and parameters.
- Parses blocks and nested statements.
- Maintains operator precedence for expressions.

4.3 Semantic Analysis

Semantic analysis checks the meaning of a program after syntax has been recognized. It ensures that the program is valid according to language rules.

- Rejects duplicate declarations in the same scope.
- Rejects undeclared identifiers.
- Checks assignment compatibility.
- Checks condition types in if, while, and for.
- Checks function argument count and argument types.
- Checks return values against function return types.

4.4 Symbol Table Design

The symbol table stores declarations and supports nested scopes. It can enter a new scope, exit a scope, declare a symbol, resolve a symbol, and produce a snapshot for printing.

- Variables, parameters, and functions are stored as symbols.
- Inner scopes are searched before outer scopes.
- Duplicate declarations are checked within the current scope.

4.5 Three Address Code Generation

Three Address Code is generated during parsing and semantic analysis. It uses simple instructions, temporary variables, and labels to represent program logic.

- Temporary variables are named t1, t2, t3, and so on.
- Labels are named L1, L2, L3, and so on.
- Control-flow statements are represented using conditional and unconditional jumps.
- Function calls are represented using param and call instructions.

4.6 TAC Optimization Passes

- Constant folding is performed while expressions are parsed, so literal-only expressions such as 2 + 3 * 4 are replaced by their computed value.
- Dead-code elimination is performed after TAC generation. Instructions following an unconditional return or goto are removed until a reachable label or function boundary is found.

5. Project Architecture and File Structure

The project is separated into header files in the include folder and implementation files in the src folder. This organization keeps class declarations separate from the implementation logic.

File / Folder	Purpose
Makefile	Builds the compiler, runs tests, and cleans generated files.
README.md	Explains build, run, test commands, supported features, and example TAC.
include/diagnostics.hpp and src/diagnostics.cpp	Define and implement error collection and printing.
include/lexer.hpp and src/lexer.cpp	Define and implement lexical analysis.
include/parser.hpp and src/parser.cpp	Define and implement parsing, semantic analysis, and TAC emission.
include/symbol_table.hpp and src/symbol_table.cpp	Define and implement symbol declaration, lookup, and scoping.
include/tac.hpp and src/tac.cpp	Define and implement Three Address Code storage and printing.
include/token.hpp and src/token.cpp	Define token types and readable token names.
include/types.hpp and src/types.cpp	Define the type system and compatibility rules.
tests/	Contains valid and invalid Mini C programs for testing.

6. Sample Inputs and Outputs

This section shows representative input programs and the compiler output. The exact temporary variable and label names may vary depending on parsing order, but the structure remains the same.

6.1 Sample 1: Basic Expressions and If-Else

Input file: tests/valid_basic.c

```
int main() {
    int a = 10;
    int b = 2 + 3 * 4;
    float ratio = a / 2.0;
    bool ok = ratio > 4.5 && a != 0;
```

```
    if (ok) {
        a = a + b;
    } else {
        a = a - 1;
    }

    return a;
}
```

Expected output summary:

- Program is accepted.
- Tokens are printed for keywords, identifiers, literals, operators, and punctuation.
- Symbol table contains main, a, b, ratio, and ok.
- TAC contains declarations, assignments, arithmetic operations, conditional jump, labels, and return.

```
Example TAC excerpt:
func main
declare int a
a = 10
declare int b
# constant folded 3 * 4 -> 12
# constant folded 2 + 12 -> 14
b = 14
declare float ratio
t1 = a / 2.0
ratio = t1
declare bool ok
t2 = ratio > 4.5
t3 = a != 0
t4 = t2 && t3
ok = t4
ifFalse ok goto L1
t5 = a + b
a = t5
goto L2
L1:
t6 = a - 1
a = t6
L2:
return a
endfunc main
Result: accepted
```

6.2 Sample 2: Functions

Input file: tests/valid_functions.c

```
int square(int n) {
    return n * n;
}
```

```
float average(int a, int b) {
   float total = a + b;
   return total / 2.0;
}

int main() {
   int x = square(5);
   float y = average(x, 7);
   if (y >= 10.0) {
      x = x + 1;
   }
   return x;
}
```

Expected output summary:

- Program is accepted.
- Symbol table stores function names, parameter counts, return types, and local variables.
- TAC includes func, param_in, param, call, return, and endfunc instructions.

Example TAC excerpt:

```
func square
param_in n
t1 = n * n
return t1
endfunc square
func average
param_in a
param_in b
declare float total
t2 = a + b
total = t2
t3 = total / 2.0
return t3
endfunc average
func main
declare int x
param 5
t4 = call square, 1
x = t4
declare float y
param x
param 7
t5 = call average, 2
y = t5
t6 = y >= 10.0
ifFalse t6 goto L1
t7 = x + 1
x = t7
L1:
return x
```

```
endfunc main
Result: accepted
```

6.3 Sample 3: Invalid Undeclared Identifier

Input file: tests/invalid_undeclared.c

```
int main() {
   int x = 4;
   y = x + 1;
   return x;
}
```

Expected output summary:

- The compiler rejects the program.
- Diagnostic reports use of undeclared identifier 'y'.

6.4 Sample 4: Invalid Duplicate Declaration

Input file: tests/invalid_duplicate.c

```
int main() {
   int x = 1;
   float x = 2.0;
   return x;
}
```

Expected output summary:

- The compiler rejects the program.
- Diagnostic reports duplicate declaration of 'x'.

6.5 Sample 5: Invalid Type Mismatch

Input file: tests/invalid_type_mismatch.c

```
int main() {
   int x = 10;
   char letter = 'a';
   x = "compiler";
   if ("not a condition") {
      letter = letter + 1;
   }
   return x;
}
```

Expected output summary:

- The compiler rejects the program.
- Diagnostic reports invalid assignment from string to int.
- Diagnostic reports that the if condition must be numeric or bool.

6.6 Sample 6: Dead Code Elimination

Input file: tests/valid_dead_code.c

```
int main() {
    int x = 10;
    return x;

    x = x + 99;
    int never_used = x * 2;
    return never_used;
}
```

Expected output summary:

- Program is accepted because unreachable statements are still syntactically valid.
- The generated TAC keeps the reachable return and removes the instructions that cannot execute.

```
Example TAC excerpt:
func main
declare int x
x = 10
return x
# dead code eliminated 6 unreachable instruction(s)
endfunc main
```

7. Testing and Validation

Testing is performed using the make test command. The test command runs valid files that should be accepted and invalid files that should be rejected.

```
make test
```

The Makefile executes the compiler on the following tests:

- The test suite also includes tests/valid_dead_code.c to demonstrate the new dead-code elimination optimization.

Test File	Expected Result	Purpose
valid_basic.c	**Accepted**	Basic declarations, expressions, if-else, return
valid_functions.c	**Accepted**	Function definitions, function calls, parameters, return types
valid_control_flow.c	**Accepted**	while loop, for loop, loop updates
valid_arrays.c	**Accepted**	Array declaration, indexing, assignment, and loop-based access
invalid_undeclared.c	**Rejected**	Use of undeclared variable
invalid_type_mismatch.c	**Rejected**	Invalid assignment and invalid condition type
invalid_duplicate.c	**Rejected**	Duplicate declaration in the same scope

8. Limitations and Future Enhancements

8.1 Limitations

- The compiler is a frontend only and does not generate executable machine code.
- The supported C subset is limited.
- Pointers, structures, unions, and full preprocessor support are not implemented.
- Standard C library functions are not implemented.
- Only lightweight optimizations such as constant folding and simple TAC dead-code elimination are supported.
- Advanced error recovery is limited.

8.2 Future Enhancements

- Add Abstract Syntax Tree generation.
- Add LLVM IR or assembly code generation.
- Add support for pointers and structures.
- Improve error recovery and diagnostic messages.
- Add more advanced optimization passes such as constant propagation, control-flow-graph analysis, and loop optimizations.
- Add register allocation and backend code generation.
- Support more complete C grammar features.

9. Conclusion

The Mini C Compiler project successfully demonstrates the essential frontend phases of compiler construction. It reads Mini C source programs, performs lexical analysis, parses the token stream using recursive-descent parsing, checks semantic correctness through a symbol table and type system, and generates Three Address Code as an intermediate representation.

The project includes meaningful valid and invalid test cases, making it possible to verify both successful compilation and error detection. Overall, the compiler provides a practical implementation of important concepts such as tokenization, grammar parsing, scope handling, type checking, diagnostics, intermediate code generation, and constant folding.

References / Project Files Used

- Provided Mini C compiler source code and project files.
- Makefile and README description.
- Header files: diagnostics.hpp, lexer.hpp, parser.hpp, symbol_table.hpp, tac.hpp, token.hpp, and types.hpp.
- Source files: diagnostics.cpp, lexer.cpp, main.cpp, parser.cpp, symbol_table.cpp, tac.cpp, token.cpp, and types.cpp.
- Test files from the tests folder.